# HealMesh — Implementation & Execution Plan

Status: Draft v1 Basis: Direct-API core (Option B). Sequenced by phase gates from `docs/PRD.md`, with component build order from `docs/TDD.md` §"How the components map to build order." Milestones are ordered, not calendar-dated — each phase's Definition of Done (DoD) is the actual gate, since accuracy work (benchmarking) can't be time-boxed honestly in advance.

Legend: **[Owner]** = team focus area from PRD §"Team roles" (Core, Infra, Middleware/Surface).

---

## Phase 0 — Foundations & decisions (before any feature code)

**Goal:** Resolve every open question that would otherwise block or bias Phase 1 work, and stand up the reusable scaffolding.

| Task | Owner | Notes |
|---|---|---|
| Rename `prism` → `healmesh` across namespace, Helm chart, docs | Infra | Mechanical; do first so nothing downstream references the old name |
| LLM provider selection (Claude vs GPT-4o-class) | Core | Evaluate structured-output reliability, latency, cost/incident on a small pilot set; document the decision and rationale in TDD §3.7 — do not leave implicit |
| `healmesh-k8s` language decision (Python vs Go) | Infra | Record decision + rationale |

| | | |
|---|---|---|
| | | before Phase 1 kickoff |
| Adapt `backend/CONTRACTS.md` → formal incident/diagnosis JSON schema (Pydantic/JSON Schema) | Core | Starting point preserved per TDD §6, not redesigned from zero |
| Adapt `backend/migrations/001_remediation_incidents.sql` → Postgres schema for incidents/diagnoses/actions/approvals | Core | Starting point preserved; add append-only triggers in the same migration, not later |
| Retire `backend/workflows/prism-incident.json` (n8n) and OpenClaw integration | Core/Infra | Confirm no remaining runtime dependency before Phase 1 code review |
| Set up CI: dependency scanning, container hardening, linting, type-checking gates | Infra | Non-root containers, minimal base images from day one |
| Bootstrap cert-manager + internal TLS | Infra | Cheap now, expensive to retrofit — per TDD §5 |

**DoD for Phase 0:** LLM provider and language decisions are documented with rationale; namespace/chart renamed and building cleanly; CI pipeline runs on every PR; no code depends on n8n or OpenClaw.

---

## Phase 1 — Read-only diagnosis (the actual v1, per PRD §6)

**Goal:** Detect the five canonical failure types, produce a diagnosis, deliver to Slack. Nothing executes. This is the phase where the security-critical parser gets built and proven in isolation, before a cluster connection exists.

### 1a. healmesh-core, built and tested standalone first (no cluster needed)

| Task | Owner | DoD |
|---|---|---|
| Incident schema validator | Core | Rejects and logs any malformed input; 100% branch coverage on validation failure paths |
| Diagnosis prompt engine (shared template, per TDD §3.5) | Core | Produces correct prompt for all 5 failure types from fixture incidents; log truncation/sanitization applied before every LLM call |
| Direct LLM API integration (structured output / tool-use, per chosen provider) | Core | Returns machine-parseable structured response for 100% of a fixture set; no free-text JSON hoping |
| Remediation Action Parser | Core | Closed enum only; unparseable → `NONE`; full unit test coverage including adversarial/malformed LLM outputs (see `docs/TESTING.md` §Parser) |
| Audit logger (Postgres, no update/delete methods, DB trigger) | Core | Attempting update/delete at the code level raises immediately; DB trigger independently verified |
| Rate limiting + cost caps on LLM calls | Core | Verified under simulated incident-storm load test |

### 1b. healmesh-k8s, watcher-only (read path)

| Task | Owner | DoD |
|---|---|---|
| Event Watcher (client-go or kubernetes-client, watch API, no polling) | Infra | Detects all 5 canonical failure types (TDD §3.4) against a local kind cluster with injected failures |
| Incident creation pipeline | Infra | Confirmed zero write imports/permissions in this package (CI-enforced) |
| RBAC: `get/list/watch` only, target namespace scoped | Infra | Applied service account cannot patch/update/delete/create/exec — verified by a negative RBAC test |
| Internal TLS to healmesh-core | Infra | Verified via network trace / cert validation, not just config presence |

### 1c. Surface — Slack (read-only diagnosis delivery)

| Task | Owner | DoD |
|---|---|---|
| Slack webhook delivery of diagnosis (root cause, confidence, suggested manual command) | Surface | End-to-end: injected failure → Slack message, p95 < 30s (PRD FR-3) |
| HMAC signature verification on any inbound Slack interaction | Surface | Spoofed/unsigned request rejected before any payload parsing |

**DoD for Phase 1 (= PRD FR-1 through FR-5):** All 5 failure types detected; diagnosis delivered to Slack within 30s p95; append-only audit log has a record for every incident; zero write capability exists anywhere in the deployed system (verifiable via RBAC and code review, not just intent).

---

## Phase 1.5 — Benchmark & demo (explicit gate before Phase 2 per PRD §10)

**Goal:** Prove the diagnosis is actually good before building anything that acts on the cluster.

| Task | Owner | DoD |
|---|---|---|
| Build benchmark set: 30–50 hand-built + real-incident cases across all 5 failure types | Core | Set is versioned, reviewed for representativeness across failure types |
| Run benchmark, measure accuracy per failure type | Core | Reported honestly regardless of where it lands (PRD non-functional: Honesty) — no cherry-picking |
| Record end-to-end demo: real failure injected → Slack diagnosis → manual fix → recovery confirmed | Core/Infra | Recorded artifact exists and is reviewable |

**DoD for Phase 1.5 (hard gate):** Benchmark accuracy ⩾ 80% overall is the target (PRD §9); if it lands below target, Phase 2 does not start until either the diagnosis approach is improved or the shortfall is explicitly accepted and documented by the team, not silently waived. Weak spots per failure type are visible in the report, not hidden.

---

## Phase 2 — Gated remediation, SCALE only (per PRD §6 Phase 2)

**Goal:** Exactly one remediation action, human-approved every time, with automatic rollback on failure.

| Task | Owner | DoD |
|---|---|---|
| Approval Workflow Engine: routes `SCALE` to Slack approve/reject, records approver identity + timestamp in the same write transaction | Core | No auto-approve path exists in code, not just config |
| Remediation Executor: accepts only the closed-enum `SCALE` action, separate service account/identity from the watcher | Infra | Type signature makes passing a raw string a compile/type error, not a runtime check |
| Hardcoded namespace denylist enforced in executor code | Infra | Test written and passing *before* execution logic is merged (per AGENTS.md convention) |
| Pre-action snapshot storage | Infra | Snapshot exists and is retrievable before every write, verified in test |
| Post-remediation health checker + automatic rollback | Infra | Simulated failed health check triggers rollback in test environment, confirmed via snapshot restore |
| RBAC: add `patch` on Deployment scale subresource, scoped to explicitly enabled namespaces only | Infra | Negative test confirms no other verb/resource is grantable via this path |
| Migrate credentials to Vault (or equivalent) — must land before this phase ships (TDD §4) | Infra | No write-capable credential exists in a plain Kubernetes Secret or env var by the time Phase 2 deploys |

**DoD for Phase 2 (= PRD FR-7, FR-8):** `SCALE` executes only after recorded human approval; every execution has a pre-action snapshot and a post-action health check; rollback is proven, not assumed; namespace denylist cannot be bypassed by any HealPolicy-equivalent config at this stage (CRD doesn't exist yet — a simple allowlist config is fine per TDD §3.3).

---

## Phase 3 — Expanded remediation + policy (per PRD §6 Phase 3)

**Goal:** Add `PATCH`, `REDEPLOY`, `HELM_UPGRADE` one at a time, each independently gated; introduce HealPolicy CRD; replace Slack-only with a Dashboard once volume justifies it.

| Task | Owner | DoD |
|---|---|---|
| Add `PATCH` action type, approval-gated by default | Core/Infra | Full parser + executor test coverage before enabling; no widening of enum acceptance logic itself |
| Add `REDEPLOY` action type, approval-gated by default | Core/Infra | Same as above |
| Add `HELM_UPGRADE` action type, approval-gated by default | Core/Infra | Same as above, plus Helm release access scoped per-namespace |
| HealPolicy CRD: per-namespace allowed actions, auto-approve vs. human sign-off, excluded namespaces | Infra | New namespace defaults to diagnosis-only until explicitly opted in (secure-by-default check, tested) |
| Dashboard: read incidents/diagnoses/audit history; write limited to approve/reject | Surface | SSO/OIDC (or per-user session auth) wired up *before* the first approve button ships, not after |
| Approver identity logging on every dashboard approval | Surface | Logged in the same transaction as the approval, not a bolted-on afterward step |

**DoD for Phase 3 (= PRD FR-9, FR-10):** Each new action type ships with full test coverage and defaults to approval-required; HealPolicy CRD correctly restricts behavior per namespace and cannot be bypassed by a missing or malformed policy (fails closed, not open); Dashboard approval path is fully audited.

---

## Phase 4 — Platform expansion (per PRD §6 Phase 4)

**Goal:** SDK for non-cluster-native incident submission, multi-cluster support, enterprise features — only once there's real demand, per PRD non-goals for earlier phases.

| Task | Owner | DoD |
|---|---|---|
| SDK (Python/Node) with scoped per-integration API tokens | Surface | Compromised integration token can submit incidents only, never read other tenants' data or trigger execution |

| | | |
|---|---|---|
| Multi-cluster support | Infra | Explicitly deferred until a second real cluster/customer exists — do not build speculatively |
| Enterprise: SSO, on-prem LLM option, blockchain-anchored audit log | Core/Infra | Built only once a real customer's compliance requirement is driving it (per TDD §7, PRD §6) |

**DoD for Phase 4:** Each sub-feature ships only against a real, named driver (a specific integration request, a specific customer requirement) — not spec-built ahead of demand.

---

## Cross-phase rules that apply throughout

- No phase begins its execution-capable work until the previous phase's DoD is met and documented — this mirrors the PRD's explicit "Phase 2+ scope is blocked until Phase 1.5 benchmark and demo exist" gate (PRD §10 Risks).
- Every task that adds a write capability requires its corresponding test to exist in the same PR (see `docs/TESTING.md`), not as a follow-up ticket.
- Security controls are part of each task's Definition of Done, not a separate hardening pass — if a task's DoD above doesn't already say so explicitly, treat the relevant control from `docs/TDD.md`'s security-controls sections as implied and required.

---

## Suggested milestone review points

1. **End of Phase 0:** decisions documented, CI green, no legacy n8n/OpenClaw dependency.
2. **End of Phase 1:** live Slack diagnosis demo against a real injected failure, zero write capability anywhere in the system.
3. **End of Phase 1.5:** benchmark report published internally (and, per PRD open questions, a decision made on whether/how it's shared externally).
4. **End of Phase 2:** one real approved SCALE execution with rollback, on a disposable test cluster, recorded end-to-end.
5. **End of Phase 3:** HealPolicy CRD demo showing a namespace explicitly opted into an action type it wasn't allowed to use by default.
6. **Phase 4:** revisit only when a concrete external driver exists.